

VendorPulse — Demo Guide, System Flow & Agent Hosting

Companion to the Client Meeting Prep Pack. Three things: what to show on screen, how the system actually works (in plain terms), and the "are these hosted Foundry agents?" answer.

Prepared for the Zensar team · for the prospective-client review · Confidential

1. What to demo on screen

Key thing to get straight first: MAF (Microsoft Agent Framework) has no UI of its own — it is a code SDK that runs inside your FastAPI backend. There is no "MAF screen" to open. When the client says "show us the Azure AI side," the visuals you actually have are **(a) the Azure AI Foundry portal** and **(b) your own VendorPulse app**. Don't go hunting for a MAF dashboard — it doesn't exist.

Show these three, in this order of impact:

#	What to show	The message it sends	Notes
1	The VendorPulse app (React UI) — especially the ApprovalPanel	"The AI drafts, a human approves, every step is logged." This is the governance money-shot.	Also walk the WorkflowProgressBar (12 states), CycleDetail tabs (modules A–F), and ActionLog (audit trail).
2	Azure AI Foundry portal (ai.azure.com) — your project, Deployments → gpt-4o , the Playground , and Monitoring/Tracing	"The model runs on Foundry, inside your tenant."	Proves it's live and reachable.
3	<i>(for the AI architect)</i> Run the PoC live + show the agent code	"We own every prompt; the approval gate is real."	Run <code>python poc_scheduling_foundry.py --gate</code> to show the gated <code>send_invites</code> tool being refused , then open <code>app/agents/scheduling_agent.py</code> .

*Do NOT open the Foundry "Agents" playground and imply that's our agent. Our design is **MAF SDK in-process** — the agents are our code, not a hosted Foundry agent — so that tab will be empty/unrelated and a sharp architect will catch the mismatch. Say instead: "our agents are our own code calling Foundry's Responses API — Foundry serves the model; we own the agent logic."*

*If you only have two minutes: show the **ApprovalPanel in the app** + the **gpt-4o deployment in the Foundry portal**. That covers "what it does" and "where the AI runs."*

2. How the system works — in plain terms

The restaurant analogy (the one to remember)

Piece	Restaurant role
Browser / React app	The dining room & menu — what the customer sees and clicks.

Piece	Restaurant role
FastAPI backend	The kitchen + head chef + manager — holds the recipes (rules), the safe (secrets), the ledger (database); decides what's allowed and coordinates everyone. Customers never enter the kitchen.
MAF agent	A specialist who <i>drafts</i> a nicely-worded "special of the day" when asked.
Azure AI Foundry (GPT-4o)	The creative assistant the specialist consults for wording.
Microsoft Graph	The courier that actually sends the email / books the calendar.
PostgreSQL	The ledger where everything is recorded.
The approval gate	The manager signing off before any dish leaves the kitchen.

What FastAPI does, and why it's necessary

FastAPI is the **server in the middle**. It's necessary because the browser cannot be trusted to do these four jobs:

- Hold the secrets.** The Foundry credential, the Graph certificate, the database password must *never* reach the browser (anyone could read them). Only a server can hold them safely — **FastAPI is the trust boundary**.
- Run the deterministic rules.** The **WorkflowEngine** (the 12 states), slot ranking, and score validation — the logic that must be reliable and auditable — live here as plain code. This *is* the governance backbone.
- Orchestrate everything.** It runs the MAF agents, calls Foundry, calls Graph, and reads/writes the database. It's the conductor.
- Enforce the approval gate.** The rule "don't send until a human approves" is enforced *server-side*, where the browser can't bypass it.

Why FastAPI specifically: it's Python — the same language the MAF/AI SDKs live in — it's fast and async, and it auto-validates data and auto-generates API docs.

The flow, end to end — "Prepare invite" example

1. User clicks **"Prepare invite"** in the React app.
2. Browser sends an HTTP request to **FastAPI** (POST /scheduling/.../draft).
3. FastAPI **checks who you are** (Entra login) and that the workflow is **in the right state**.
4. FastAPI hands the job to the **MAF agent**, which calls **Foundry (GPT-4o)** and its tools (rank slots, draft invite text).
5. Foundry returns a **draft** (ranked slots + invite wording), flagged **"requires approval."** **Nothing is sent.**
6. FastAPI **logs the run** (audit) and returns the draft to the browser.
7. The **ApprovalPanel** shows the draft → a human **reviews and clicks Approve**.
8. Browser calls a **separate** FastAPI route (POST /.../send-invites).
9. FastAPI re-checks the state, then **deterministically calls Microsoft Graph** to actually send the invite — **the AI is not involved in this step**.
10. The workflow **advances one state**; everything is logged.

***The one-liner:** "The browser shows; FastAPI decides, orchestrates, and enforces the rules; Foundry only drafts wording; Graph only sends after a human approves."*

The crucial point for a governance client: **drafting (AI) and sending (deterministic) are two separate requests with a human in between.** The AI literally cannot send anything — it doesn't have the tool, and the send only fires from a different route after approval.

3. Agent hosting — where we are vs the Hosted-Agents roadmap

The client may ask: "are these hosted Foundry agents, and could they be?" Here is the precise answer.

Today: MAF SDK, in-process

Your agents are Python code (app/agents/*.py) running **inside your FastAPI backend** on the VM. Foundry's only job is to **serve the model** (GPT-4o via the Responses API). Foundry is **not** running your agent — your server is. This was a deliberate choice.

"Foundry agent" means three different things — don't blur them

What it is	Who runs your agent code	Who owns the prompt	Status
1. Foundry Prompt / Agent-Service agents (built in the portal UI)	Foundry	Foundry assembles it (you don't fully own it)	GA
2. Foundry Hosted Agents (your MAF code, hosted by Foundry)	Foundry (managed endpoint)	You (your code)	Public preview
3. MAF SDK in-process — current design	Your backend	You	GA

Can you make them hosted Foundry agents? — Yes, by design

The relevant option is **#2, Foundry Hosted Agents**: Foundry runs your *same MAF agent code* as a managed endpoint, giving managed scaling, a per-agent Entra identity, session state, and built-in tracing — so you no longer manage the agent runtime yourself. **No rewrite of the agent logic is needed.** Three honest caveats:

- It's public preview, not GA.** For a Shell production system that must clear IRM / Shell.AI / TRB, depending on a preview feature is usually a blocker — which is exactly why the baseline is in-process.
- In-process was a deliberate choice** for full prompt ownership, the deterministic fallback (ENABLE_LLM=false), the approval gate, and data residency (it runs where your backend runs). Hosted Agents trades some control for less infra burden.
- It's not a one-click move.** The agents call your services/tools (slot ranking, Graph, the database). To host them, those tools/data must be reachable *from the hosted runtime* (networking + identity), and the agent must be packaged for it. The **approval gate still works** (the agent returns a draft either way; the deterministic send stays in the app) — but it's real work, not a toggle.

What to say in the room

*"Today the agents run in-process via the MAF SDK and call Foundry's Responses API — we chose that for full prompt ownership, the deterministic fallback, and the approval gate, with no preview dependency, which matters for your security review. **Foundry Hosted Agents can host the exact same MAF code as a managed endpoint** — that's our planned hosting upgrade once it's GA, and it requires no change to the agent logic."*

That answer shows you (a) understand the option, (b) made a defensible decision, and (c) have a forward path — far stronger than looking like you "didn't use the managed thing."

For today: don't scramble to stand up a hosted agent — it's preview, it'd be rushed, and it contradicts the documented decision. Present it as the **roadmap**: in-process now → Hosted Agents when GA, same code.

4. Positioning — why in-process MAF over portal Prompt/Agent-Service agents

Use this if the client is leaning toward building agents in the Foundry portal. The move: don't frame it "their idea vs ours" — frame it as "which Foundry path fits a governance product," and offer the managed option they actually want (Hosted Agents) so they don't feel overruled.

GA facts to have ready (verified, Microsoft)

- bullet Foundry Agent Service (the platform): **GA since March 2026**.
- bullet Foundry Hosted Agents (Foundry runs *your* MAF code on managed compute): **public preview now; GA expected early July 2026**. BYO-VNet / no public egress supported; preview regions include **Sweden Central (EU)** — but **West Europe is not yet confirmed** (a data-residency check for Shell).
- bullet Content safety / XPIA filters are **Foundry-level** — they apply to our calls **regardless** of orchestration approach.

The 30-second headline pitch

*"We're 100% on Microsoft Foundry either way — same GPT-4o, same Responses API, same Azure tenant. The only question is who orchestrates the agent: the Foundry portal, or our own code on the Microsoft Agent Framework. For a **governance product that has to satisfy IRM 3.492 and the EU AI Act**, we deliberately chose MAF in-process — because it lets us **own every prompt token, prove it to your auditors, and make the human-approval gate architectural rather than configured**. Portal-built agents hand that control to the platform. We'd rather give your IRM and security teams the version that's easiest to certify — and keep the door open to Foundry's managed Hosted Agents the moment they go GA in July, with zero code change."*

The reframe to open with

They favour portal agents because it *feels* like the blessed, low-effort, managed path. Agree with the instinct, then redirect:

*"You're right to want the Microsoft-managed route — and there are **two** of them. The portal **Prompt/Agent-Service agents**, where Foundry assembles and drives the prompt; and **Hosted Agents**, where Foundry runs your agent code on managed compute. For a governance product, the managed option you actually want is **Hosted Agents** — it gives you the managed platform without surrendering prompt control. That's GA in early July, and our code drops straight into it."*

That turns "portal vs ours" into "the right managed option vs the wrong one."

The five arguments — each tied to what Shell cares about

- bullet **1. Auditability (IRM 3.492 + EU AI Act)**. With our code, the *exact* prompt is in Git — versioned, diffable, shown to auditors verbatim. Portal agents have Foundry assemble the instruction context, so you can't put "the prompt" in front of an auditor and say "this, exactly, every time." Prompt transparency isn't a nice-to-have here; it's the control.
- bullet **2. The human-approval gate is the product**. Agent-Service agents are built to *act* — call tools, complete tasks autonomously. Our safety model is the opposite: the agent **drafts**, a human **approves**, then a deterministic route sends. In-process, the agent literally **has no send tool**. On the portal you'd fight the act-default to re-create that gate, and re-prove it every release.
- bullet **3. Deterministic-first is what makes the AI safe**. Decisions — slot ranking, score validation, workflow transitions — are plain code, not the LLM. The portal model assumes the agent drives the process; adopting it dilutes the "AI only drafts text, code makes every decision" story your VMO trusts.
- bullet **4. It's easier for Shell to certify, not harder (IRM 3.3)**. Our code in your repo is *more* reviewable than portal-configured agents — your scanners see all of it, it's unit-testable, and no logic hides in a portal config outside your CI. One artifact, fully in your pipeline.

5. It keeps more options open, including Microsoft's. In-process code lifts onto **Hosted Agents** (managed, GA July) as a ~1-week hosting swap — same code. Portal agents can't be lifted out; you'd lock into that config. The LLMProvider abstraction also keeps Anthropic selectable. We're preserving optionality, not rejecting Microsoft.

Objection handling — if they say X, you say Y

If they say...	You say...
"Why not just use the managed portal agents?"	"Because <i>managed</i> should mean Hosted Agents — managed compute for our code — not handing Foundry control of the prompt. Managed benefits, without the audit cost."
"Isn't building it yourselves more risk / more code?"	"Less risk for <i>you</i> . It's GA, hits the 4-week plan, every line is in your CI and reviewable. Portal agents are actually a rewrite that risks the timeline."
"Microsoft recommends Agent Service."	"And we're on it — via the Responses API. Microsoft ships the Agent Framework SDK <i>for exactly this</i> : teams needing full prompt control and custom approval flows. We're using the Microsoft tool built for governance."
"We want content safety / guardrails."	"Those are Foundry-level — content filters and XPiA apply to our calls regardless of orchestration. You lose no safety by orchestrating in code."
"What about scale / ops burden?"	"Steady internal workload — a right-sized service handles it, and Hosted Agents gives managed autoscale as the July fast-follow, same code."

The close — give them a win

*"Our recommendation: **ship on MAF in-process now** — GA, on-timeline, fully auditable, the approval gate built into the architecture — and we've **pre-engineered the path to Foundry Hosted Agents** so you get Microsoft's managed compute as a fast-follow the week it's GA, with no rebuild. You get the managed platform you're asking for and the audit control your IRM team requires. The portal Prompt-agent route is the one path that costs you both prompt ownership and your approval gate — which is why we'd steer around it."*

The psychology: you're not saying "no" to their managed-platform desire — you're offering the *right* managed option on a roadmap. Far easier for a Product Owner and AI architect to accept than a flat rejection.

Sources (GA timing)

- devblogs.microsoft.com/foundry/foundry-agent-service-ga/
- devblogs.microsoft.com/foundry/introducing-the-new-hosted-agents-in-foundry-agent-service-secure-scalable-compute-built-for-agents/
- learn.microsoft.com/en-us/azure/foundry/agents/concepts/hosted-agents
- devblogs.microsoft.com/foundry/whats-new-in-microsoft-foundry-apr-2026/